

Unit 3: Knowledge Representation and Probabilistic Reasoning

SubUnit 3.3: Propositional and Predicate Logic

Kiran Bagale

IOE, Thapathali Campus

December, 2025

Why Logic?

Logic provides a formal, precise language for representing knowledge and reasoning.

Key Advantages:

- **Formal semantics:** Unambiguous meaning
- **Compositionality:** Build complex statements from simple ones
- **Sound inference:** Guaranteed correct conclusions
- **Well-studied:** Centuries of mathematical foundation

Two Main Types:

- ① **Propositional Logic:** Deals with true/false propositions
- ② **Predicate Logic (First-Order Logic):** Adds objects, relations, quantifiers

Reference: Russell & Norvig (2020), Chapter 7-8

Propositional Logic: Syntax

Basic Elements:

- **Atomic propositions:** $P, Q, R, S, \dots$ (symbols representing facts)
- **Logical connectives:** Combine propositions

Symbol	Name	Meaning	Example
$\neg$	Negation	NOT	$\neg P$ (not P)
$\wedge$	Conjunction	AND	$P \wedge Q$ (P and Q)
$\vee$	Disjunction	OR	$P \vee Q$ (P or Q)
$\Rightarrow$	Implication	IF...THEN	$P \Rightarrow Q$ (if P then Q)
$\Leftrightarrow$	Biconditional	IF AND ONLY IF	$P \Leftrightarrow Q$ (P iff Q)

Examples

- P : "It is raining"
- Q : "The ground is wet"
- $P \Rightarrow Q$: "If it is raining, then the ground is wet"

$$\begin{aligned}
 \textit{Sentence} &\rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence} \\
 \textit{AtomicSentence} &\rightarrow \textit{True} \mid \textit{False} \mid P \mid Q \mid R \mid \dots \\
 \textit{ComplexSentence} &\rightarrow (\textit{Sentence}) \mid [\textit{Sentence}] \\
 &\mid \neg \textit{Sentence} \\
 &\mid \textit{Sentence} \wedge \textit{Sentence} \\
 &\mid \textit{Sentence} \vee \textit{Sentence} \\
 &\mid \textit{Sentence} \Rightarrow \textit{Sentence} \\
 &\mid \textit{Sentence} \Leftrightarrow \textit{Sentence}
 \end{aligned}$$

OPERATOR PRECEDENCE : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

Figure: A BNF (Backus–Naur Form) grammar of sentences in propositional logic, along with operator precedences, from highest to lowest.

Propositional Logic: Semantics

Truth Values: Each proposition is either **True (T)** or **False (F)**

Truth Tables: Define meaning of connectives

Negation & Conjunction

P	Q	$\neg P$	$P \wedge Q$
T	T	F	T
T	F	F	F
F	T	T	F
F	F	T	F

Disjunction & Implication

P	Q	$P \vee Q$	$P \Rightarrow Q$
T	T	T	T
T	F	T	F
F	T	T	T
F	F	F	T

Important Note

$P \Rightarrow Q$ is false **only when** P is true and Q is false!

Reference: Russell & Norvig (2020), Section 7.4.2

Standard logical equivalences

Two sentences are logically equivalent iff true in same models: $\alpha \equiv \beta$ iff $\alpha \models \beta$ and $\beta \models \alpha$

$$\begin{aligned}(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge\end{aligned}$$

Figure: The symbols α , β , and γ stand for arbitrary sentences of propositional logic.

Inference Rules

- **Modus Ponens:** $\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$ means that, whenever any sentences of the form $\alpha \Rightarrow \beta$ and α are given, then the sentence β can be inferred. For example, if $(\text{WumpusAhead} \wedge \text{WumpusAlive}) \Rightarrow \text{Shoot}$ and $(\text{WumpusAhead} \wedge \text{WumpusAlive})$ are given, then Shoot can be inferred.
- **And-Elimination:** $\frac{\alpha \wedge \beta}{\alpha}$ from a conjunction, any of the conjuncts can be inferred. For example, from $(\text{WumpusAhead} \wedge \text{WumpusAlive})$, WumpusAlive can be inferred.
- **Resolution:** $\frac{\alpha \vee \beta, \neg\beta \vee \gamma}{\alpha \vee \gamma}$

Example

$P \Rightarrow Q, P \vdash Q$

(If it rains, the ground is wet. It rains. Therefore, the ground is wet.)

Defining a Proof problem

- **INITIAL STATE:** the initial knowledge base.
- **ACTIONS:** the set of actions consists of all the inference rules applied to all the sentences that match the top half of the inference rule.
- **RESULT:** the result of an action is to add the sentence in the bottom half of the inference rule.
- **GOAL:** the goal is a state that contains the sentence we are trying to prove.

Monotonicity

Final property of logical systems, which says that the set of entailed sentences can only increase as information is added to the knowledge base.

For any sentences α and β , if $KB \models \alpha$ then $KB \wedge \beta \models \alpha$.

Resolution

The idea of resolution is simple: if we know that

- p is true or q is true
- and we also know that p is false or r is true
- then it must be the case that q is true or r is true.

Reasoning is formalized in the **Resolution Tautology**:

$$(p \vee q) \wedge (\neg p \vee r) \rightarrow (q \vee r)$$

Applying Resolution in a Proof

- 1 Express hypotheses and conclusion as a product of sums (conjunctive normal form), such as those that appear in the Resolution Tautology.
- 2 Each maxterm in the CNF of the hypothesis becomes a clause in the proof.
- 3 Apply the resolution tautology to pairs of clauses, producing new clauses.
- 4 If all the clauses of the conclusion are produced, proved.

Proof with Resolution

Given the following hypotheses:

- 1 If it rains, Joe brings his umbrella: $r \rightarrow u$
- 2 If Joe has an umbrella, he doesn't get wet: $u \rightarrow \neg w$
- 3 If it doesn't rain, Joe doesn't get wet: $\neg r \rightarrow \neg w$

Prove that: Joe doesn't get wet ($\neg w$)

Converting to CNF

We first put each hypothesis in CNF:

- 1 $r \rightarrow u \equiv (\neg r \vee u)$
- 2 $u \rightarrow \neg w \equiv (\neg u \vee \neg w)$
- 3 $\neg r \rightarrow \neg w \equiv (r \vee \neg w)$

Resolution Proof

Use resolution on the hypotheses to derive the conclusion ($\neg w$):

- | | | |
|---|----------------------|--------------------|
| ① | $\neg r \vee u$ | Premise |
| ② | $\neg u \vee \neg w$ | Premise |
| ③ | $r \vee \neg w$ | Premise |
| ④ | $\neg r \vee \neg w$ | L1, L2, resolution |
| ⑤ | $\neg w \vee \neg w$ | L3, L4, resolution |
| ⑥ | $\neg w$ | L5, idempotence |
| ⑦ | QED | |

Key Concepts in Propositional Logic

1. **Model:** An assignment of truth values to propositions
2. **Satisfiability:** A sentence is **satisfiable** if there exists a model where it's true
3. **Validity (Tautology):** A sentence that is true in **all** models
 - Example: $P \vee \neg P$ (always true)
4. **Entailment:** $KB \models \alpha$
 - Knowledge base KB **entails** sentence α if α is true in all models where KB is true

Example

KB: $\{P \Rightarrow Q, P\}$

Does $KB \models Q$?

Yes! In all models where both $P \Rightarrow Q$ and P are true, Q must also be true.

Reference: Russell & Norvig (2020), Section 7.4.3

Limitations of Propositional Logic

Problem: Cannot express relationships between objects

Example Scenario

Statement: "All students are intelligent"

Propositional Logic:

$S_1 \Rightarrow I_1$ (If person 1 is a student, then person 1 is intelligent)

$S_2 \Rightarrow I_2$ (If person 2 is a student, then person 2 is intelligent)

$S_3 \Rightarrow I_3$ (If person 3 is a student, then person 3 is intelligent)

$\vdots$

Need separate proposition for **each person!** Not scalable!

What's Missing:

- Cannot represent **objects** (students, people)
- Cannot represent **properties** and **relations**
- Cannot use **variables** and **quantifiers**

Reference: Russell & Norvig (2020), Section 8.1

Horn and Definite Clauses

Horn Clause

Definition: A Horn Clause is a disjunction of literals with **at most one positive literal**.

General Form: $\neg p_1 \vee \neg p_2 \vee \dots \vee \neg p_n \vee q$

which is equivalent to the implication: $p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q$

Key Property: At most one positive literal (0 or 1)

Definite Clause

Definition: A Definite Clause is a Horn Clause with **exactly one positive literal**.

General Form: $\neg p_1 \vee \neg p_2 \vee \dots \vee \neg p_n \vee q$

or in implication form: $p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q$

where q is called the **head** and $p_1 \wedge p_2 \wedge \dots \wedge p_n$ is the **body**.

Key Property: Exactly one positive literal (must be 1)

Horn Clause - Examples

Example 1: $\neg \text{Raining} \vee \neg \text{Cold} \vee \text{Snow}$

- Equivalent to: $\text{Raining} \wedge \text{Cold} \Rightarrow \text{Snow}$
- Has one positive literal: Snow

Example 2: $\neg \text{Student} \vee \neg \text{Studies} \vee \text{Passes}$

- Equivalent to: $\text{Student} \wedge \text{Studies} \Rightarrow \text{Passes}$
- Has one positive literal: Passes

Example 3: $\neg \text{Fire} \vee \neg \text{Oxygen}$

- Equivalent to: $\text{Fire} \wedge \text{Oxygen} \Rightarrow \text{false}$
- Has zero positive literals (goal clause)

Definite Clause - Examples

Example 1: $\neg \text{Human}(X) \vee \text{Mortal}(X)$

- Implication form: $\text{Human}(X) \Rightarrow \text{Mortal}(X)$
- Body: $\text{Human}(X)$, Head: $\text{Mortal}(X)$
- Meaning: All humans are mortal

Example 2: $\neg \text{Bird}(X) \vee \neg \text{CanFly}(X) \vee \text{Flies}(X)$

- Implication form: $\text{Bird}(X) \wedge \text{CanFly}(X) \Rightarrow \text{Flies}(X)$
- Body: $\text{Bird}(X) \wedge \text{CanFly}(X)$, Head: $\text{Flies}(X)$
- Meaning: Birds that can fly, do fly

Example 3 (Fact): $\text{Cat}(\text{Whiskers})$

- A fact is a definite clause with no body (empty body)
- Meaning: Whiskers is a cat

Horn vs Definite Clauses - Comparison

Property	Horn Clause	Definite Clause
Positive literals	At most 1 (0 or 1)	Exactly 1
Can be a fact?	Yes	Yes
Can be a goal?	Yes	No
Example	$\neg p \vee \neg q$	$\neg p \vee q$
Use in Prolog	Some types	Primary form

Key Insight: All definite clauses are Horn clauses, but not all Horn clauses are definite clauses (e.g., goal clauses with 0 positive literals are Horn but not definite).

Why They're Important in AI

They are crucial for:

- Knowledge representation
- Logic programming (like Prolog)
- Forward/backward chaining algorithms
- Deriving new facts from existing rules and data
- **Efficient Reasoning:** Their structure allows for powerful, efficient inference algorithms (like Selective Linear Definite (SLD) resolution)
- **Knowledge Representation:** Ideal for encoding rules and facts in logic-based AI
- **Logic Programming:** Forms the foundation for languages like Prolog, enabling declarative programming where you state what is true, not how to compute it

First-Order Logic: Beyond Propositions

First-Order Logic (FOL) extends propositional logic with:

- ① **Objects:** Constants (John, 2, UCB)
- ② **Relations:** Predicates describing properties
 - *Student(John)* - John is a student
 - *GreaterThan(5, 3)* - 5 is greater than 3
- ③ **Functions:** Map objects to objects
 - *father(John)* - John's father
 - *sqrt(16)* - square root of 16
- ④ **Variables:** x, y, z - represent any object
- ⑤ **Quantifiers:** Express "for all" and "there exists"
 - $\forall$ - Universal quantifier (for all)
 - $\exists$ - Existential quantifier (there exists)

Reference: Russell & Norvig (2020), Chapter 8

First-Order Logic: Syntax

Terms (refer to objects):

- Constants: *John*, 5, *Stanford*
- Variables: x, y, z
- Functions: *father*(x), *age*(*John*), *sqrt*(25)

Atomic Sentences:

- Predicate applied to terms: *Student*(*John*), *Loves*(*Mary*, x)
- Equality: $x = y$, *father*(*John*) = *Bill*

Complex Sentences: Built using connectives and quantifiers

- $\neg \textit{Student}(\textit{John})$ - John is not a student
- $\textit{Student}(x) \wedge \textit{Smart}(x)$ - x is a student and smart
- $\forall x \textit{Student}(x) \Rightarrow \textit{Smart}(x)$ - All students are smart
- $\exists x \textit{Student}(x) \wedge \textit{Genius}(x)$ - There exists a student who is a genius

Reference: *Russell & Norvig (2020), Section 8.2*

Combining the best of formal and natural languages

Language	Ontological Commitment (What exists in the world)	Epistemological Commitment (What an agent believes about facts)
Propositional logic	facts	true/false/unknown
First-order logic	facts, objects, relations	true/false/unknown
Temporal logic	facts, objects, relations, times	true/false/unknown
Probability theory	facts	degree of belief $\in [0, 1]$
Fuzzy logic	facts with degree of truth $\in [0, 1]$	known interval value

Figure: Formal languages and their ontological and epistemological commitments.

Quantifiers: Universal and Existential

Universal Quantifier ($\forall$): "For all"

Examples

- $\forall x \text{ Human}(x) \Rightarrow \text{Mortal}(x)$
"All humans are mortal"
- $\forall x, y \text{ Parent}(x, y) \Rightarrow \text{Loves}(x, y)$
"All parents love their children"

Existential Quantifier ($\exists$): "There exists"

Examples

- $\exists x \text{ Student}(x) \wedge \text{Smart}(x)$
"There exists a student who is smart"
- $\exists x \text{ King}(x) \wedge \text{Lives}(x, \text{Castle})$
"There exists a king who lives in a castle"

Nested Quantifiers

$\forall x \exists y \text{ Loves}(x, y)$ - "Everyone loves someone"
 $\exists y \forall x \text{ Loves}(x, y)$ - "There is someone loved by everyone"

Reference: Russell & Norvig (2020), Section 8.2.3

Quantifier Properties and Equivalences

De Morgan's Laws for Quantifiers:

Original	Equivalent
$\neg \forall x P(x)$ "Not everyone has P"	$\exists x \neg P(x)$ "Someone doesn't have P"
$\neg \exists x P(x)$ "No one has P"	$\forall x \neg P(x)$ "Everyone doesn't have P"

Common Patterns:

- $\forall x P(x) \Rightarrow Q(x)$ - "All P's are Q's"
- $\exists x P(x) \wedge Q(x)$ - "Some P is Q"
- $\forall x P(x) \vee Q(x)$ - "Everything is P or Q"

Watch Out!

$\forall x P(x) \vee Q(x)$ is NOT the same as $(\forall x P(x)) \vee (\forall x Q(x))$

Reference: Rich et al. (2009), Section 6.2

First-Order Logic: Complex Examples

Representing Complex Knowledge:

Family Relationships

- $\forall x, y \text{ Parent}(x, y) \wedge \text{Male}(x) \Rightarrow \text{Father}(x, y)$
"If x is a parent of y and x is male, then x is father of y"
- $\forall x, y, z \text{ Parent}(x, y) \wedge \text{Parent}(x, z) \wedge (y \neq z) \Rightarrow \text{Sibling}(y, z)$
"If x is parent of both y and z (and they're different), then y and z are siblings"

Mathematical Statements

- $\forall x (x > 0) \Rightarrow \exists y (y = \sqrt{x})$
"Every positive number has a square root"
- $\forall x, y (x < y) \Rightarrow \exists z (x < z \wedge z < y)$
"Between any two numbers, there exists another number"

Reference: Russell & Norvig (2020), Section 8.2.4

Inference in Propositional and First-Order Logic

Inference Methods:

Propositional Logic:

- Model checking (truth tables)
- Resolution
- Forward chaining
- Backward chaining

Complexity:

- Decidable
- NP-complete
- Efficient for small KBs

First-Order Logic:

- Unification
- Resolution (with unification)
- Forward chaining (Datalog)
- Backward chaining (Prolog)

Complexity:

- Semi-decidable
- More expressive
- Slower inference

Trade-off

FOL is more expressive but computationally more expensive than propositional logic

Reference: *Russell & Norvig (2020), Chapters 7.5, 9.2*

Models for first-order logic

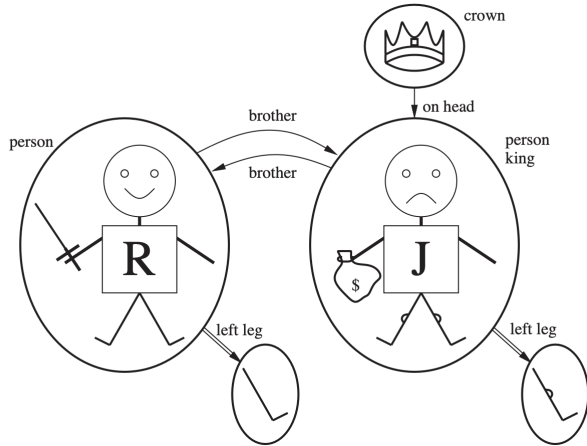


Figure: A model containing five objects, two binary relations, three unary relations (indicated by labels on the objects), and one unary function, left-leg.

The Knowledge-Engineering Process

Knowledge Engineering Projects

Vary widely in content, scope, and difficulty.

- **Identify the task:** Analogous to the PEAS process for designing agents
- **Assemble the relevant knowledge:** knowledge acquisition
- **Decide on a vocabulary of predicates, functions, and constants:** Domain level into logic level
- **Encode general knowledge about the domain**
- **Encode a description of the specific problem instance**
- **Pose queries to the inference procedure and get answers**
- **Debug the knowledge base**

Summary: Propositional vs. First-Order Logic

Aspect	Propositional Logic	First-Order Logic
Basic units	Atomic propositions	Objects, relations, functions
Variables	None	Yes (x, y, z)
Quantifiers	None	$\forall, \exists$
Example	$Rain \Rightarrow Wet$	$\forall x \text{ } Rain(x) \Rightarrow Wet(x)$
Expressiveness	Limited	High
Decidability	Decidable	Semi-decidable
Complexity	NP-complete	Undecidable (general)
Best for	Simple rules, Boolean reasoning	Complex domains, relationships

Key Takeaway: Choose based on expressiveness needs vs. computational constraints

References: Russell & Norvig (2020), Ch 7-8; Rich et al. (2009), Ch 6